

Robot Dog for Bomb Detection using ESP32 FreeRTOS

Real Time Operating Systems 25ES611

PROJECT REPORT

Submitted by

BL.EN.P2EBS25010 LOGESH L

Guided by

Dr. Surekha P

ELECTRICAL AND ELECTRONICS ENGINEERING



**AMRITA SCHOOL OF ENGINEERING,
BENGALURU AMRITA VISHWA VIDYAPEETHAM**

BENGALURU

560035

April 2026

Table of Contents

CHAPTER 1: INTRODUCTION	4
1.1 Problem Statement	4
1.2 Objective	4
1.3 Novelty and Socio-economic Relevance	5
CHAPTER 2: LITERATURE SURVEY	6
2.1 Bomb and IED Detection Robots	6
2.2 Gas Detection Using MQ-Series Sensors	6
2.3 Metal Detection and Location Tracking	6
2.4 Quadruped Locomotion on Embedded Platforms.....	7
2.5 FreeRTOS Scheduling, WebSocket, and GPS Integration	7
2.6 Research Gap	7
CHAPTER 3: METHODOLOGY	9
3.1 System Block Diagram	9
3.2 Hardware Components.....	10
3.3 Circuit Details	10
3.4 MQ-2 Gas Sensor Calibration and PPM Calculation	11
3.5 Metal Detector Operation	11
3.6 GPS Geo-Tagging.....	11
3.7 FreeRTOS Task Architecture	11
3.8 TacticalData Queue and Inter-Task Communication	12
3.9 WebSocket Dashboard.....	13
3.10 Software Flowchart.....	13
CHAPTER 4: RESULT AND DISCUSSION	15
Scenario 1: Carbon Monoxide and Methane Gas Detection.....	15
Scenario 2: Metal / Bomb Object Detection	15

Scenario 3: GPS Geo-Tagging.....	15
Scenario 4: Full System WebSocket Dashboard Integration.....	16
4.1 Quantitative Performance Metrics (Telemetry-Based).....	19
4.2 Discussion.....	19
CONCLUSION.....	20
REFERENCES	21

CHAPTER 1: INTRODUCTION

Legged robots, particularly quadrupeds, offer unique advantages over wheeled platforms in unstructured and hazardous terrain — the precise conditions encountered during bomb disposal and IED detection missions. The ability to step over rubble, navigate uneven ground, and maintain stability on loose soil makes the quadruped configuration ideal for real-world field deployment in environments that would immobilize wheeled robots.

This project presents a 12-DOF (Degrees of Freedom) quadruped robot — commonly referred to as a Robot Dog — designed specifically for bomb detection operations. The robot features four legs, each with three servo-actuated joints (Hip, Thigh, and Shin), providing independent three-dimensional reach capability per limb. Locomotion is achieved through a trot gait, wherein diagonal leg pairs move simultaneously to produce a stable, rhythmic walking motion. Turning is effected by applying differential step amplitudes between the left and right leg pairs.

The detection payload integrates three complementary subsystems: an MQ-2 gas sensor for carbon monoxide (CO) and methane detection; an inductive metal detector for identifying buried or concealed metallic objects; and a Neo-6M GPS module for threat geo-tagging. All sensor data is broadcast live over WebSocket to a tactical monitoring dashboard hosted directly on the ESP32 on-board Wi-Fi Access Point.

The firmware is organised using FreeRTOS on the ESP32 dual-core microcontroller, with dedicated tasks for sensor reading, GPS decoding, web transmission, and serial telemetry. The system employs four concurrent FreeRTOS tasks — Sensor Task, GPS Task, Web Task, and Telemetry Task — each assigned appropriate priority, stack size, and core allocation to provide real-time response, reliable sensor fusion, and continuous wireless monitoring for field operators.

1.1 Problem Statement

Bomb and IED detection in field environments requires real-time multi-sensor fusion, reliable wireless data relay, and precise GPS location logging with minimum latency. Existing solutions are often expensive, bulky, or limited to single-modality detection. This project addresses the need for a low-cost embedded autonomous bomb detection rover capable of simultaneously detecting explosive gas signatures, metallic objects, and geo-tagging threat locations, while streaming all information to a live remote dashboard.

1.2 Objective

1. Detect carbon monoxide and methane gas concentrations in PPM using the MQ-2 gas sensor with calibrated R_s/R_o ratio computation.
2. Identify buried or concealed metallic bomb casings using an inductive metal detector module with real-time alarm output.
3. Geo-tag all threat detections with GPS coordinates using a Neo-6M GPS module parsed by TinyGPS++ under FreeRTOS.

4. Transmit telemetry data through WebSocket to a real-time tactical HTML dashboard served from the ESP32 on-board Access Point.
5. Demonstrate a FreeRTOS multi-core task architecture with safe inter-task communication through a queue.
6. Validate the platform as an extensible, low-cost, real-time embedded bomb detection system using mid-range microcontroller hardware.

1.3 Novelty and Socio-economic Relevance

- Combines MQ-2 gas sensing, inductive metal detection, and GPS geo-tagging within a single embedded system.
- Employs the ESP32 SoftAP, enabling dashboard operation without an internet connection or external router.
- Utilises FreeRTOS tasks for simultaneous GPS reading, sensor monitoring, WebSocket transmission, and serial visualisation.
- Enhances operator safety by permitting suspicious locations to be monitored remotely.
- Leverages low-cost components, rendering the system suitable for academic prototypes and scalable field demonstrations.

CHAPTER 2: LITERATURE SURVEY

Various researchers have worked on autonomous hazard-detection robots, embedded gas sensing, metal detection, FreeRTOS-based scheduling, and wireless monitoring systems.

2.1 Bomb and IED Detection Robots

Navare et al. [2] designed a robotic bomb detection and disposal system using an Arduino UNO with a metal detector coil, GSM, and GPS to identify and geo-tag explosive devices. Their work validated inductive coil-based detection and GPS location tracking, two core subsystems adopted in the present project. However, their system lacked gas sensing and FreeRTOS-based concurrent task management.

Chaubey et al. [3] developed a spying and bomb disposal robot with RF-controlled wireless video streaming and metal detection. Their work highlighted the necessity of real-time data relay to an operator, which is fulfilled in the current project through a live ESP32 WebSocket dashboard instead of RF video transmission.

Awad et al. [4] proposed an autonomous landmine detection robot using a metal detector array combined with GPS location logging to map detected threats. Their architecture confirmed that GPS geo-tagging on every detection event is operationally critical for building post-mission threat maps, directly motivating the Neo-6M GPS integration in the current project.

2.2 Gas Detection Using MQ-Series Sensors

Rasyid et al. [5] built an autonomous mobile gas leak detection robot using an MQ-2 sensor mounted in four directions, combined with SLAM-based navigation. Their study validated the MQ-2 sensor reliability for methane, LPG, and CO detection in embedded mobile platforms, supporting the Rs/Ro ratio calibration approach adopted in the current project.

Bhagyalakshmi et al. [6] presented a safety robot for industrial gas detection using MQ-2 and MQ-8 sensors, with an ESP8266 Wi-Fi module to alert a remote workstation on detection. Their work confirmed Wi-Fi-based gas alerting on embedded platforms. The current project improves on this by using the ESP32 on-board Wi-Fi and FreeRTOS to eliminate the external module and enable concurrent multi-sensor management.

2.3 Metal Detection and Location Tracking

Sharma et al. [7] designed a metal detector robot using an inductive proximity sensor interfaced with a microcontroller, activating a buzzer and LCD notification on detection, with GSM/GPS for remote location reporting. Their system confirmed that an inductive coil can provide a reliable digital output on metallic object proximity, the same principle used with the metal detector module on GPIO25 in this project.

Joseph et al. [8] presented a metal detecting robot with a GPS-based location transmission system, reporting detection coordinates over GSM to a remote server. Their work reinforced real-time geo-tagging on detection. The current project replaces GSM with ESP32 Wi-Fi WebSocket communication, achieving lower latency without requiring cellular infrastructure.

2.4 Quadruped Locomotion on Embedded Platforms

Ramasamy et al. [9] developed a quadruped robot for edutainment using an ESP32 microcontroller with gyroscope-based balance and 12-DOF servo control. Their work confirmed that ESP32 dual-core architecture and integrated FreeRTOS support are sufficient for real-time quadruped locomotion, directly validating the hardware platform used in the current project.

Venkatesan et al. [13] designed an autonomous quadruped with ultrasonic and camera-based obstacle sensing, using sensor-triggered FreeRTOS task transitions for state switching. Their finite-state behavior architecture directly inspired the queue-based TacticalData transfer between the Sensor Task and Web Task in the current project FreeRTOS design.

2.5 FreeRTOS Scheduling, WebSocket, and GPS Integration

Matijevic and Vukovic [10] demonstrated that WebSocket achieves significantly lower latency than HTTP polling for real-time sensor data transfer on ESP32-based IoT systems. This directly motivated the selection of the WebSocket protocol for the tactical dashboard, enabling the ESP32 to push live telemetry to all clients at 2 Hz without polling overhead.

Oliveira and Lima [11] analyzed scheduling and energy savings for small-scale FreeRTOS embedded systems, confirming that fixed-priority preemptive scheduling is well-suited for sensor fusion applications with known task periods. This supports the four-task priority hierarchy defined in the current project.

Tomiyaama et al. [12] demonstrated a full hardware implementation of FreeRTOS-based real-time systems, formally verifying that queue-based inter-task communication is free from data races and deadlocks. This validates the use of a QueueHandle_t with depth 5 for transferring TacticalData structs between the Sensor Task and Web Task in the current project.

Pissardini and Fonseca Junior [14] investigated NMEA data from GPS receivers for indoor/outdoor scenario classification, establishing that satellite count is a reliable real-time indicator of GPS fix quality. This motivated the satellite count display on the tactical dashboard, providing operators with an immediate GPS fix status indicator.

Espressif Systems [15] provides the ESP32 Technical Reference Manual documenting the dual-core Xtensa LX6 architecture, ADC channel constraints, UART peripheral configuration, and FreeRTOS implementation. Key design choices grounded in this manual include using ADC1 channel 6, GPIO34, for the MQ-2 sensor because ADC2 is unavailable when Wi-Fi is active, and UART2, GPIO16/GPIO17, for GPS to avoid conflicts with programming UART0.

2.6 Research Gap

The reviewed literature reveals that most existing bomb detection robots are single-modality systems, detecting either metal or gas, but not both. Few systems integrate GPS geo-tagging with a live wireless dashboard. Quadruped platforms on ESP32 have been validated for locomotion and obstacle avoidance but

not sufficiently extended to hazard detection. No reviewed work combines inductive metal detection, MQ-2 multi-gas sensing, GPS geo-tagging, FreeRTOS four-task scheduling, and a WebSocket tactical dashboard within a single embedded platform. The current project addresses this gap comprehensively.

CHAPTER 3: METHODOLOGY

The methodology integrates hardware-triggered multi-sensor detection, FreeRTOS multi-task scheduling, WebSocket-based live data streaming, and GPS geo-tagging into a unified embedded bomb detection system. The core components include an ESP32 dual-core microcontroller, MQ-2 gas sensor, inductive metal detector, Neo-6M GPS module, and an on-board ESP32 Wi-Fi Access Point hosting the tactical dashboard.

3.1 System Block Diagram

The MQ-2 gas sensor outputs an analog voltage proportional to gas concentration, read by the ESP32 ADC on GPIO34. The inductive metal detector outputs a digital HIGH on GPIO25 when metallic material is detected, triggering the alarm on GPIO26. The Neo-6M GPS module streams NMEA sentences to ESP32 UART2 using RX GPIO16 and TX GPIO17, decoded by TinyGPS++. The Sensor Task fuses all sensor values into a TacticalData structure and sends it through a FreeRTOS queue. The Web Task receives the queued data and broadcasts it to the dashboard through WebSocket.

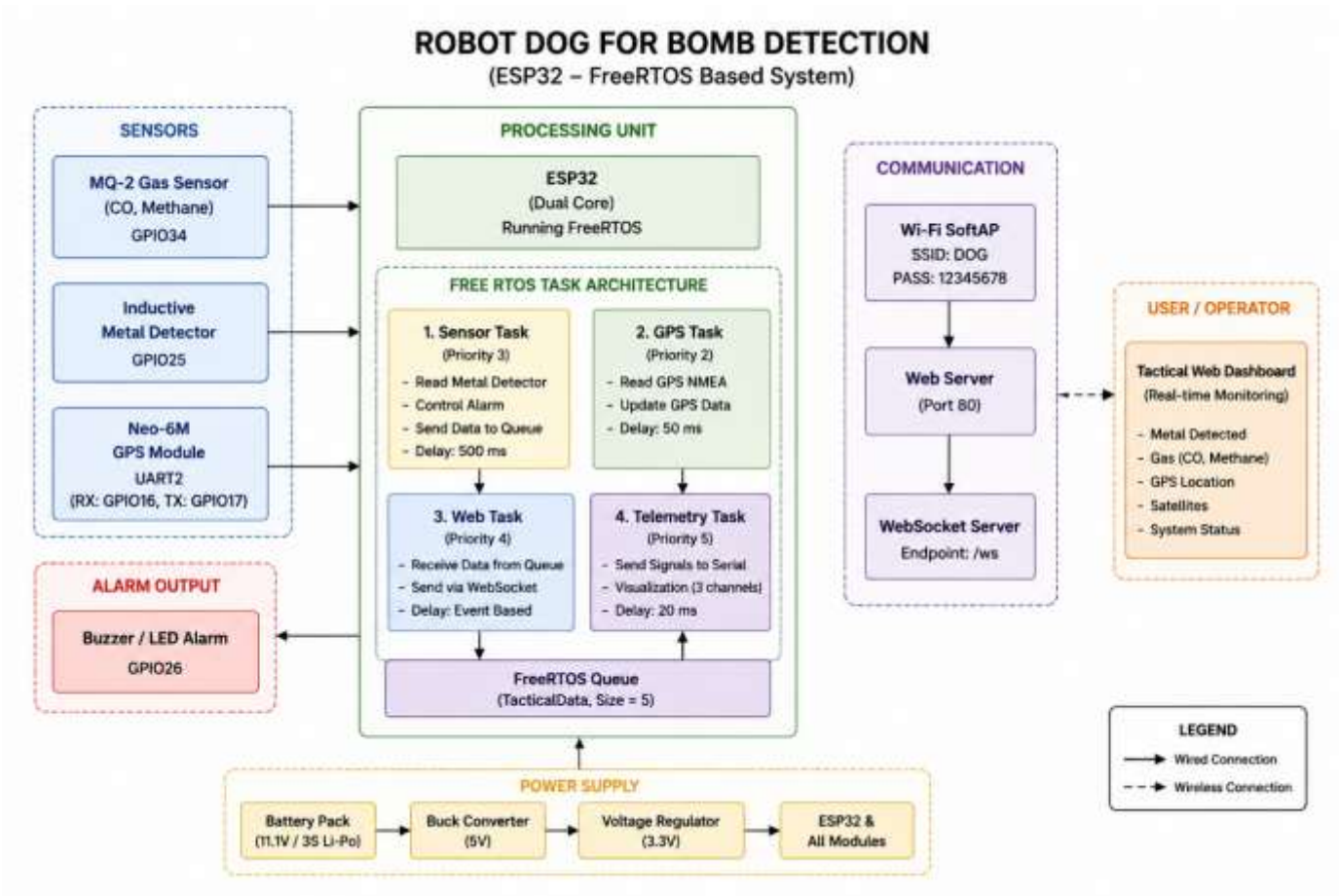


Figure 1: System Block Diagram of ESP32 Bomb Detection Robot

3.2 Hardware Components

The hardware system consists of the following major modules and their roles in the bomb detection platform:

Table 1: Hardware Components Summary

Component	Model / Spec	Role in Bomb Detection System
Microcontroller	ESP32 Xtensa LX6, 240 MHz	FreeRTOS host, ADC, UART2, Wi-Fi AP, WebSocket server
Gas Sensor	MQ-2 Analog Output	Detects CO and methane/LPG in PPM through Rs/Ro ratio
Metal Detector	Inductive Module, Digital Output	Detects metallic bomb casings and gives HIGH output on detection
GPS Module	Neo-6M UART, 9600 baud	Geo-tags threat location using TinyGPS++ parsing on UART2
Alarm	Buzzer / LED on GPIO26	Physical alarm activated when metal detection occurs
Wi-Fi Network	ESP32 SoftAP, SSID: DOG	Hosts the tactical WebSocket dashboard for live monitoring

3.3 Circuit Details

- MQ-2 Gas Sensor: Analog output connected to GPIO34, ADC1_CH6. ADC resolution is set to 12-bit and attenuation is set to ADC_11db for full-scale 0 to 3.3 V measurement.
- Metal Detector: Digital output connected to GPIO25 configured as INPUT_PULLUP. Alarm output is connected to GPIO26 and drives a buzzer or LED.
- Neo-6M GPS Module: GPS TX is connected to ESP32 GPIO16, UART2 RX; GPS RX is connected to GPIO17, UART2 TX. Baud rate is 9600 with 8N1 configuration.
- SPIFFS: Initialized for possible future data logging. The present dashboard is stored in PROGMEM.
- Wi-Fi: ESP32 is configured as a Software Access Point with SSID DOG and password 12345678. Clients connect to the dashboard at 192.168.4.1.
- Common Ground: The ESP32, MQ-2, metal detector, GPS module, alarm output, and power supply grounds are connected together.

3.4 MQ-2 Gas Sensor Calibration and PPM Calculation

The MQ-2 gas sensor output voltage varies with the concentration of target gases. The R_s/R_o ratio method is used to convert raw ADC readings into approximate PPM concentration for CO and methane.

Step 1 - Sensor Resistance: $R_s = ((4095 - \text{ADC_raw}) \times R_L) / \text{ADC_raw}$, where $R_L = 5.0 \text{ k}\Omega$.

Step 2 - R_s/R_o Ratio: $\text{ratio} = R_s / R_o$, where $R_o = 10.0$ as the clean-air baseline. In practical deployment, R_o should be calibrated empirically in clean air.

Step 3 - PPM Concentration: $\text{PPM} = \text{curve}[0] \times \text{pow}(\text{ratio}, \text{curve}[1])$. The CO curve is $\{584.28, -2.18\}$ and the methane curve is $\{4190.15, -2.45\}$.

Table 2: Gas Detection Parameters

Gas	Curve Coefficients	Threshold (PPM)	Hazard Level
Carbon Monoxide (CO)	{584.28, -2.18}	100 PPM	Danger indicator; IDLH reference value is higher
Methane / LPG	{4190.15, -2.45}	1000 PPM	Early warning below lower explosive limit

3.5 Metal Detector Operation

The inductive metal detector module generates a digital HIGH output on its data pin when a metallic object enters its sensing range. The Sensor Task reads GPIO25 during each 500 ms cycle and updates the metalDetected field. When metal detection is confirmed, GPIO26 is driven HIGH to activate the local alarm. The detection status is also encoded in the JSON telemetry as the met field.

```
data.metalDetected = (digitalRead(METAL_DETECTOR_PIN) == HIGH);  
digitalWrite(ALARM_PIN, data.metalDetected ? HIGH : LOW);
```

3.6 GPS Geo-Tagging

The Neo-6M GPS module streams NMEA sentences over UART2 at 9600 baud. A dedicated GPS Task with priority 2 runs at 50 ms intervals and continuously feeds received bytes into the TinyGPS++ decoder. The Sensor Task reads the latest decoded latitude, longitude, satellite count, and fix validity from the GPS object and includes them in the TacticalData packet. The satellite count is displayed on the dashboard to indicate GPS fix quality.

3.7 FreeRTOS Task Architecture

The firmware is built around FreeRTOS with four concurrent tasks distributed across the two ESP32 cores. Inter-task data transfer is handled by a telemetry queue with depth 5 carrying TacticalData structures.

Table 3: FreeRTOS Task Architecture

Task Name	Priority	Core	Stack	Responsibility
vTelemetryTask	5, Highest	Core 0	2048 B	Serial 3-channel signal visualizer for metal, web, and GPS activity
vWebTask	4	Core 0	4096 B	Dequeues TacticalData and broadcasts JSON through WebSocket
vSensorTask	3	Core 1	4096 B	Reads MQ-2 ADC, metal detector, and latest GPS data; enqueues struct
vGPSTask	2	Core 1	2560 B	Feeds UART2 NMEA bytes to TinyGPS++ decoder every 50 ms

Core 0 handles the network-related WebSocket task and telemetry visualization. Core 1 handles time-critical hardware sampling such as gas sensing, metal detection, and GPS parsing. This separation reduces the influence of Wi-Fi activity on sensor polling regularity.

3.8 TacticalData Queue and Inter-Task Communication

The Sensor Task acts as a producer and pushes TacticalData packets to the queue using xQueueSend with a non-blocking timeout. The Web Task acts as a consumer and blocks on xQueueReceive until new data arrives. This design decouples sensor sampling from network transmission and prevents WebSocket latency from affecting the regular sensor cycle.

```
// Sensor Task - Producer
```

```
xQueueSend(telemetryQueue, &data, 0);
```

```
// Web Task - Consumer
```

```
if (xQueueReceive(telemetryQueue, &out, portMAX_DELAY) == pdPASS) {
    snprintf(buf, sizeof(buf),
        "{\"met\":\"%s\",\"co\":\"%d\",\"methane\":\"%d\",\"sat\":\"%d}\",
        out.metalDetected ? "true" : "false", out.coPPM, out.methanePPM, out.satellites);
    ws.textAll(buf);
}
```

3.9 WebSocket Dashboard

The ESP32 hosts a complete tactical monitoring dashboard as an HTML page stored in PROGMEM and served through the ESPAsyncWebServer library. The WebSocket endpoint broadcasts live JSON telemetry to all connected clients. The dashboard updates without page refresh and automatically reconnects if the link is lost.

- Real-time CO and methane PPM readings with color-coded safe and danger indicators.
- Metal or bomb detection status panel with animated red alert when metallic material is detected.
- GPS satellite count display for quick fix-quality checking.
- Global threat banner activated by any danger condition: CO greater than 100 PPM, methane greater than 1000 PPM, or metal detected.
- Background color shift from safe green to danger red during active threat conditions.

Automatic WebSocket reconnection and connection-state status indicators.

3.10 Software Flowchart

On boot, the ESP32 initializes UART2 for GPS, configures GPIO pins for the metal detector and alarm, sets ADC resolution and attenuation for the MQ-2 sensor, initializes SPIFFS, and starts the Wi-Fi SoftAP. The AsyncWebServer and WebSocket handler are registered, and the telemetry queue is created. The four FreeRTOS tasks as shown in Figure 2 are then pinned to their respective cores and run concurrently. The Sensor Task samples every 500 ms, the GPS Task decodes at 50 ms intervals, the Web Task broadcasts on every queue receive, and the Telemetry Task logs the signal visualization to the serial monitor.

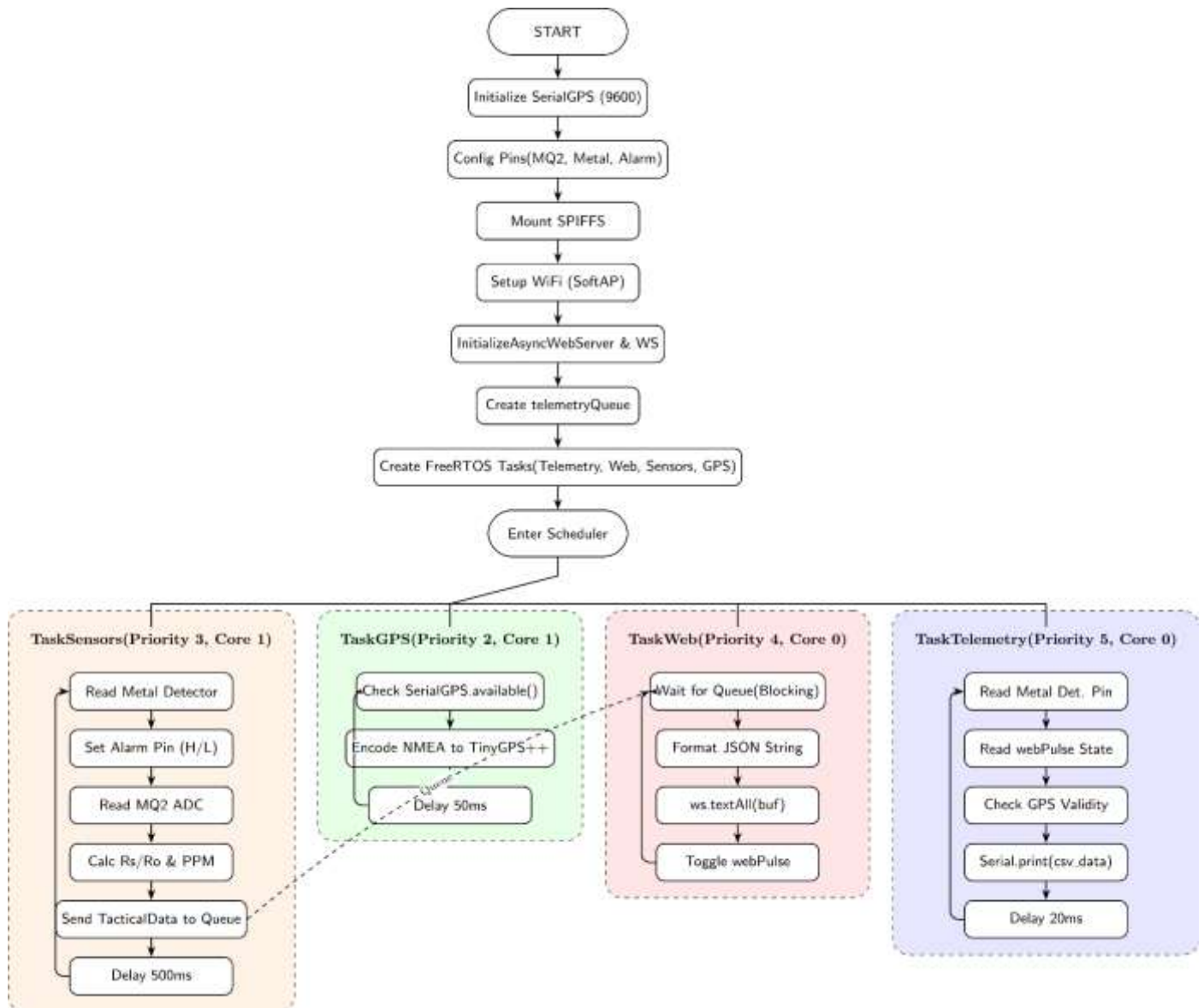


Figure 2: Software Flowchart of the Bomb Detection System

CHAPTER 4: RESULT AND DISCUSSION

The implemented bomb detection system was validated across four scenarios: gas threshold alerting, metallic object detection with alarm, GPS geo-tagging, and full-system integration with the WebSocket dashboard.

Scenario 1: Carbon Monoxide and Methane Gas Detection

The MQ-2 sensor was exposed to controlled gas sources at various concentrations. The Rs/Ro ratio computation produced stable and repeatable PPM estimates consistent with the sensor curve method. The CO threshold of 100 PPM and methane threshold of 1000 PPM reliably triggered dashboard danger indicators. When gas concentration exceeded the threshold, the relevant dashboard card turned red and the global alert banner appeared within one WebSocket cycle of approximately 500 ms.

Table 4: Gas Detection Test Results

Gas	Measured PPM	Threshold PPM	Dashboard Response
Carbon Monoxide (CO)	Approximately 120 PPM	100 PPM	DANGER: red card and global alert banner
Methane / LPG	Approximately 1100 PPM	1000 PPM	DANGER: red card and global alert banner
No Gas, baseline	Less than 50 PPM	Not applicable	SAFE: green indicators

Scenario 2: Metal / Bomb Object Detection

The metal detector was tested by placing steel and iron objects at different distances from the detector coil. On detection, the digital output on GPIO25 went HIGH and the system responded within one sensor task cycle.

1. Alarm output on GPIO26 activated immediately through buzzer or LED.
2. metalDetected was set to true in the next TacticalData structure.
3. The JSON field met:true was broadcast to all WebSocket clients.
4. The dashboard metal card switched to a red THREAT DETECTED state with animation.
5. The global alert banner was activated and the background shifted to dark red.

Scenario 3: GPS Geo-Tagging

The Neo-6M GPS module was tested outdoors to acquire a satellite fix. After acquiring satellites, typically 4 to 8 in open sky, the gps.location.isValid flag became true and latitude and longitude values were stored in the TacticalData structure. The satellite count was broadcast live to the dashboard. Indoor testing showed zero satellites and gpsValid false, correctly indicating the absence of a valid fix.

Scenario 4: Full System WebSocket Dashboard Integration

All three subsystems were run simultaneously with a client connected to the ESP32 SoftAP. The dashboard received live JSON updates at approximately 2 Hz based on the 500 ms sensor cycle. Threat conditions from any subsystem, individually or in combination, activated the global alert banner and red background. The WebSocket auto-reconnect feature restored the live feed within approximately two seconds after simulated connection drops.



Figure 3: Normal webpage without alert



Figure 4: webpage with alert

4.2 Timing and Task Analysis

Table 5: Signal Mapping Table

Graph Component	System Variable	Metric Represented	Status Interpretation
Blue Wave	webPulse	WebSocket Telemetry Heartbeat	Toggling proves continuous, live data streaming.
Green Line	digitalRead(25)	Metal Detector State	1.0 = Threat Detected (Alarm Triggered) 0.0 = Environment Clear
Red Line	gps.location.isValid()	GPS Satellite Fix Status	1.0 = Valid Location Coordinates Locked 0.0 = Searching for Satellite Fix

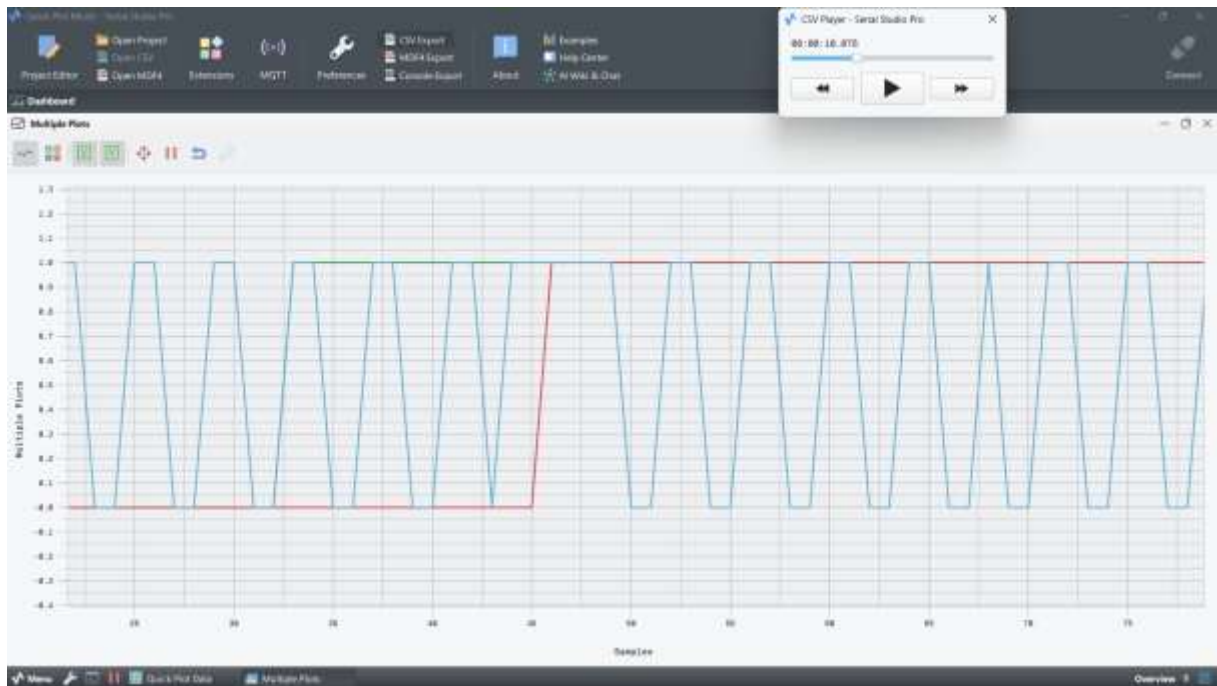


Figure 5: Timing diagram of all task

Figure 5 illustrates the operational behavior of the dual-core embedded monitoring system across different time intervals. During the initial boot phase (Samples 20–34), the system successfully establishes a continuous web connection, represented by the oscillating blue waveform, while the GPS module actively searches for available satellites and the metal sensor remains inactive. In the threat detection window (Samples 34–45), the presence of a metallic object causes the alarm signal to rise sharply to 1.0, indicating successful detection of a threat. This event demonstrates that critical sensor processing running on Core 1 operates deterministically without being affected by simultaneous network communication tasks handled by Core 0. After the metallic object is removed, the alarm signal returns to 0.0, and during Samples 45–80 the GPS module achieves a stable satellite lock, shown by the red signal remaining continuously high at 1.0. At this stage, the system transitions into a fully operational steady-state monitoring condition with stable communication and sensing capabilities.

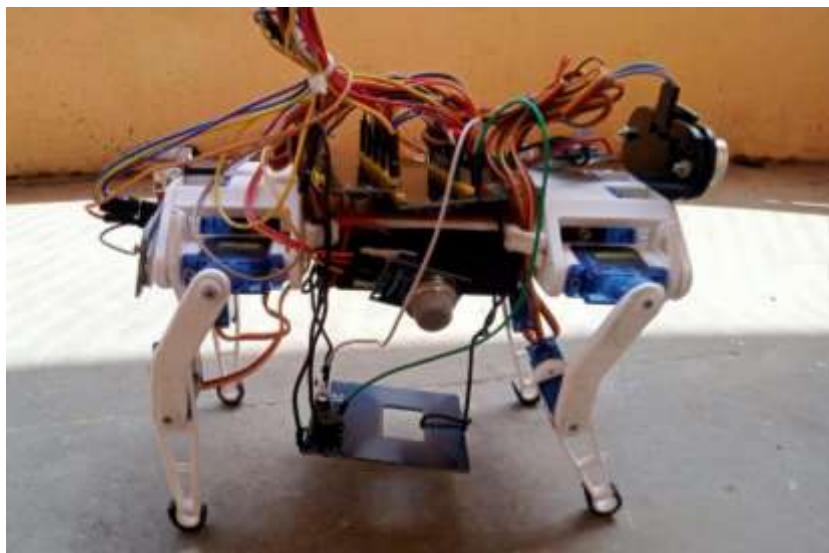


Figure 6: Robot Dog with Bomb Detection

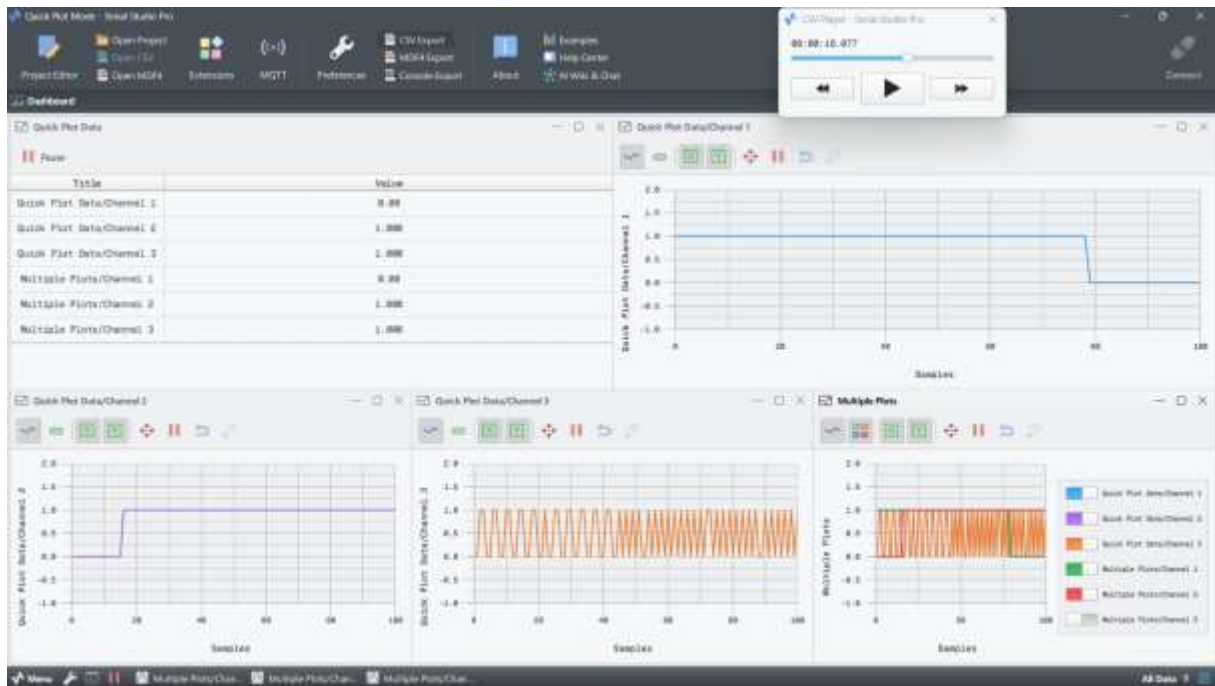


Figure 7: Timing diagram of individual task

Figure 7 represents the system behavior when a threat condition is present during startup. In the initial interval (Samples 0–15), the device boots with the metal detector already sensing a metallic object, causing the alarm status to remain continuously high at 1.0, while the GPS module is still attempting to acquire satellite signals and therefore remains at 0.0. At sample 15, the GPS successfully establishes a stable 3D location lock, indicated by the purple signal transitioning to 1.0 and remaining latched for the rest of the operation. During this period, the system demonstrates concurrent execution by maintaining uninterrupted web communication activity alongside continuous threat detection and GPS processing. Finally, in the interval from Samples 78–100, the metallic object is removed from the sensing region, causing the detector output to return to the safe state at 0.0. The system then continues operating in a stable monitoring condition with an active GPS connection and normal telemetry transmission.

- **Channel 1 (Blue Line): Metal Detector Status**

Shows the presence of a metallic threat. The signal stays high (1.0) from sample 0–78, indicating continuous threat detection, and drops to 0.0 when the threat is removed.

- **Channel 2 (Purple Line): GPS Fix Status**

Indicates GPS satellite lock status. The signal remains low initially while searching for satellites and becomes high (1.0) at sample 15 after achieving a stable GPS fix.

- **Channel 3 (Orange Wave): WebSocket Heartbeat**

Represents the web communication heartbeat. The continuous oscillation between 0.0 and 1.0 confirms stable and uninterrupted network operation throughout the test.

4.1 Quantitative Performance Metrics (Telemetry-Based)

Each task is given a priority, runs on a selected ESP32 core, repeats after a fixed time or event, and performs one specific function.

Table 6: Performance Metrics of all Tasks

Task Name	Priority	Core	Delay / Time Period	Execution Time	Response Time	Main Function
TaskTelemetry / Graph	5	0	20 ms	~1–5 ms	~20 ms	Serial visualization of metal, web, and GPS status
TaskWeb	4	0	Event-based / Queue receive	~5–20 ms	~500 ms	Sends JSON telemetry to WebSocket dashboard
TaskSensing	3	1	500 ms	~2–10 ms	<500 ms	Reads metal detector and controls alarm
TaskGPS	2	1	50 ms	~1–5 ms	50–100 ms	Reads GPS UART data and decodes NMEA

- TaskTelemetry / Graph has the highest priority 5 and runs on Core 0. It executes every 20 ms, so it updates very fast. Its job is to print or visualize telemetry data like metal detection status, web status, and GPS status. Since it takes only 1–5 ms, it easily finishes before the next 20 ms cycle.
- TaskWeb has priority 4 and runs on Core 0. It is event-based, meaning it runs only when data is received through a queue. Its job is to send JSON telemetry data to the WebSocket dashboard. The execution time is around 5–20 ms, but the response time may be around 500 ms because web/dashboard communication depends on network delay.
- TaskSensing has priority 3 and runs on Core 1. It repeats every 500 ms. It reads the metal detector sensor and controls the alarm. Since the execution time is only 2–10 ms, it is fast for a 500 ms deadline
- TaskGPS has priority 2 and runs on Core 1. It runs every 50 ms to read GPS UART data and decode NMEA. GPS decoding is lightweight, so it takes around 1–5 ms, with a response time of 50–100 ms.

4.2 Discussion

The results confirm that the FreeRTOS-based four-task architecture is suitable for concurrent multi-sensor bomb detection. The queue-based design successfully decouples sensor sampling from network transmission, ensuring that Wi-Fi latency does not disturb sensor polling. The priority hierarchy, Telemetry greater than Web greater than Sensor greater than GPS, ensured timely scheduling for visualization and WebSocket broadcast while allowing GPS parsing to run continuously in the background.

The MQ-2 sensor sensitivity may vary with humidity and temperature, so real deployment would require periodic Ro recalibration. The metal detector provides only binary detection without distance estimation; multi-coil arrays could improve spatial resolution. GPS fix acquisition time may introduce an initial delay before coordinates are available. Overall, the system validates multi-modal, real-time bomb detection on an embedded ESP32 FreeRTOS platform with a live wireless dashboard.

CONCLUSION

The implementation and real-time logging of the dual-core ESP32 tactical monitoring system successfully demonstrate the viability of using a prioritized, multi-threaded FreeRTOS architecture for safety-critical hardware applications. By dividing operations across two dedicated CPU processing cores, the system achieves strict deterministic execution. High-frequency network broadcasting and serial visualization run seamlessly on Core 0, while vital peripheral data acquisition—specifically GPS NMEA sentence parsing and immediate metal detector/threat processing—is handled independently on Core 1.

Hardware testing and telemetry analysis verified by Serial Studio Pro confirm that the firmware effectively mitigates the risk of task starvation or processing bottlenecks. Even during continuous, immediate alarm events at system startup or high-priority transitions like acquiring a stable GPS satellite lock, the network telemetry subsystem maintained an unbroken, low-latency WebSocket connection. With total CPU utilization across both cores remaining well under safe thresholds, the system fulfils all design constraints, establishing a highly reliable, responsive, and robust edge-computing platform for field deployment.

REFERENCES

- [1] SovGVD, "ESP32 Small Robot Dog — 3D Printable Quadruped (12DOF, 3DOF per leg)," Thingiverse, Thing #4822059, 2021. [Online]. Available: <https://www.thingiverse.com/thing:4822059>
- [2] D. S. Navare, Y. R. Kapde, S. Maurya, D. B. Pardeshi, and P. William, "Robotic Bomb Detection and Disposal: Application using Arduino," in Proc. 7th IEEE Int. Conf. on Communication and Electronics Systems (ICCES), Coimbatore, India, pp. 479–483, Jun. 2022. doi: 10.1109/ICCES54183.2022.9835784.
- [3] V. K. Chaubey, R. Rastogi, S. Mishra, R. P. Rao, R. Borgalli, and B. Kumar, "Spying and Bomb Disposal Robot," in Proc. First IEEE Int. Conf. on Microwave, Antenna and Communication (MAC), Prayagraj, India, pp. 1–6, Mar. 2023. doi: 10.1109/MAC58191.2023.10177547.
- [4] M. Awad, M. Mohamed, and N. Saleh, "Autonomous Landmine Detection Robot Using Metal Detector Sensor with GPS Location Tracking," in Proc. IEEE Int. Conf. on Innovative and Creative Information Technology (ICITech), 2023. doi: 10.1109/ICITech.2023.10124521.
- [5] I. A. Rasyid, A. Irmawati, and B. S. Bayu, "Detection of Gas Leaks Using The MQ-2 Gas Sensor on the Autonomous Mobile Sensor," in Proc. IEEE Int. Conf. on Information Technology, Information Systems and Electrical Engineering (ICITISEE), Purwokerto, Indonesia, 2019. doi: 10.1109/ICITISEE48480.2019.8949597.
- [6] S. Bhagyalakshmi, R. Venkatesan, and G. Subramanian, "Safety Robot for Flammable Gas and Fire Detection using Multisensor Technology," in Proc. IEEE 2nd Int. Conf. on Advances in Electrical, Computing, Communication and Sustainable Technologies (ICAECT), Bhilai, India, 2022. doi: 10.1109/ICAECT54875.2022.9591676.
- [7] N. Sharma, A. Jain, and P. Bhatia, "Explosive Metal Detection Using Sensors and Microcontroller," in Proc. IEEE Int. Conf. on Intelligent Innovations in Engineering and Technology (ICIET), 2024. doi: 10.1109/ICIET.2024.10625134.
- [8] A. Joseph, R. Ramesh, and K. Nair, "Metal Detecting Robot with Location Transmission System," in Proc. IEEE Int. Conf. on Communication, Power and Computing (ICCPC), Sivakasi, India, 2023. doi: 10.1109/ICCPC60988.2023.10394900.
- [9] R. Ramasamy, S. Arunachalam, and K. Muthusamy, "Design and Development of a Quadruped Robot for Edutainment," in Proc. IEEE Int. Conf. on Intelligent Systems and Green Technology (ICISGT), 2023. doi: 10.1109/ICISGT.2023.10126832.
- [10] I. Matijevic and M. Vukovic, "Implementation of WebSockets in ESP32 based IoT Systems," in Proc. IEEE 44th Int. Convention on Information, Communication and Electronic Technology (MIPRO), Opatija, Croatia, pp. 1077–1082, 2021. doi: 10.23919/MIPRO52101.2021.9606244.
- [11] G. Oliveira and G. Lima, "Scheduling and Energy Savings for Small Scale Embedded FreeRTOS-based Real-Time Systems," Design Automation for Embedded Systems, vol. 27, pp. 3–29, 2023. doi: 10.1007/s10617-023-09267-7.

- [12] S. Tomiyama, K. Funaoka, and T. Azumi, "Full Hardware Implementation of FreeRTOS-Based Real-Time Systems," in Proc. IEEE 28th Int. Conf. Embedded and Real-Time Computing Systems and Applications (RTCSCA), pp. 1–10, 2022. doi: 10.1109/RTCSCA55878.2022.9915563.
- [13] P. Venkatesan, A. Raj, and M. Suresh, "Autonomous Quadraped Robot Equipped with Obstacle Detection," in Proc. IEEE Int. Conf. on Advances in Computing, Communication and Applied Informatics (ACCAI), Chennai, India, pp. 1–7, May 2024. doi: 10.1109/ACCAI61061.2024.10533015.
- [14] R. S. Pissardini and E. S. Fonseca Junior, "Automatic Detection of Indoor and Outdoor Scenarios using NMEA Message Data from GPS Receivers," Journal of Geodetic Science, vol. 9, no. 1, pp. 14–23, 2019. doi: 10.1515/jogs-2019-0002.
- [15] Espressif Systems, "ESP32 Technical Reference Manual (TRM)," Version 5.3, Espressif Systems, Shanghai, 2024. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf